Министерство науки и образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №8
по курсу «Логика и основы
алгоритмизации в инженерных задачах»
на тему «Обход графа в ширину»

Выполнил:

студент группы 25ВВВ1
Цеплов Е.В.
Разживалов Н.А.

Приняли:
к.т.н., доцент Юрова О.В.,
к.т.н., доцент Деев М.В.

Пенза 2026

Название

Обход графа в ширину

Цель работы

Изучить обход графа в ширину

Лабораторное задание

1. 1.Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G. Выведите матрицу на экран.
   2.Для сгенерированного графа осуществите процедуру обхода в ширину, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс queue из стандартной библиотеки C++.
   3.Реализуйте процедуру обхода в ширину для графа, представленного списками смежности.
2. 1 Для матричной формы представления графов реализуйте алгоритм обхода в ширину с использованием очереди, построенной на основе структуры данных «список», самостоятельно созданной в лабораторной работе № 3.
   2. Оцените время работы двух реализаций алгоритмов обхода в ширину (использующего стандартный класс queue и использующего очередь, реализованную самостоятельно) для графов разных порядков.

Ход работы

Задание 1:

1. Алгоритм Создание Графа

АЛГОРИТМ СозданиеГрафа(n, M)

Начало

// 1. Инициализация матрицы нулями

для i от 1 до n:

для j от 1 до n:

G[i][j] ← 0

// 2. Генерация рёбер

// Рассматриваем только пары i < j, чтобы не дублировать рёбра

для i от 1 до n:

```
        для j от i + 1 до n:
            если random(0..99) < r:
                G[i][j] ← 1
                G[j][i] ← 1    // граф неориентированный

    // 3. Вывод матрицы на экран
    вывести "    "
    для j от 1 до n:
        вывести j с шириной 3
    вывести перевод строки
    для i от 1 до n:
        вывести i с шириной 3
        для j от 1 до n:
            вывести G[i][j] с шириной 3
        вывести перевод строки
Конец
```

2. Алгоритм BFS

```
Начало
    queue Q                 // std::queue<int>
    Q.push(s)
    NUM[s] ← 1              // вершина посещена
    пока Q не пуста:
        v ← Q.front()
        Q.pop()
        вывести v
```

для u от 1 до n:

если G[v][u] = 1 и NUM[u] = 0:

Q.push(u)

NUM[u] ← 1

Конец

3. Алгоритм через список

Начало

queue Q // std::queue<int>

Q.push(s)

NUM[s] ← 1 // вершина s посещена

пока Q не пуста:

v ← Q.front()

Q.pop()

вывести v

для k от 1 до deg[v]:

u ← M[v][k]

если NUM[u] = 0:

Q.push(u)

NUM[u] ← 1

Конец

Задание 2:

1. Алгоритм BFS через собственную очередь реализованную как структура.

Начало

Q : ListQueue

InitQueue(Q)

```
    PushQueue(Q, s)
    NUM[s] ← 1
    пока не IsEmpty(Q):
        v ← PopQueue(Q)
        вывести v
        для u от 1 до n:
            если G[v][u] = 1 и NUM[u] = 0:
                PushQueue(Q, u)
                NUM[u] ← 1
Конец
```

2.

Тип/n	100	150	200	250	300
Queue C++	0.008	0.011	0.016	0.021	0.030
Queue	0.007	0.012	0.016	0.023	0.025

Вывод: обе реализации BFS имеют одинаковую асимптотику $O(n^2)$, так как работают по матрице смежности. Для графов порядка 100–300 разница во времени практически неразличима и находится в пределах погрешности измерения. std::queue обычно работает немного быстрее и стабильнее, а самодельная ListQueue даёт дополнительные накладные расходы на malloc/free. Заметная разница могла бы проявиться при больших n или при усреднении многократных замеров без учёта вывода.

Результат работы программы

Задание 1:

1. Генерация графа его матрицы и списки.

```
Выберите вариант работы:
1 - Генерировать граф
2 - Обход в ширину по матрице
3 - Обход в ширину по спискам смежности
4 - Обход в ширину по матрице с помощью
5 - Сравнение скорости
0 - Выход
Ваш выбор: 1

Размер: 4
Вероятность ребра(1-100): 78
      1  2  3  4
  1   0  1  0  0
  2   1  0  0  1
  3   0  0  0  1
  4   0  1  1  0
1. 2
2. 1 4
3. 4
4. 2 3
```

2. Процедура обхода в ширину по матрице.

```
Вероятность ребра(1-100): 78
      1  2  3  4
  1   0  1  0  0
  2   1  0  0  1
  3   0  0  0  1
  4   0  1  1  0
1. 2
2. 1 4
3. 4
4. 2 3

Выберите вариант работы:
1 - Генерировать граф
2 - Обход в ширину по матрице
3 - Обход в ширину по спискам смежности
4 - Обход в ширину по матрице с помощью очереди через список
5 - Сравнение скорости
0 - Выход
Ваш выбор: 2
1 2 4 3
```

3. Процедура обхода в ширину по спискам смежности.

```
Выберите вариант работы:
1 - Генерировать граф
2 - Обход в ширину по матрице
3 - Обход в ширину по спискам смежности
4 - Обход в ширину по матрице с помощью очереди через список
5 - Сравнение скорости
0 - Выход
Ваш выбор: 3
1 2 4 3
```

Задание 2:

1. Процедура обхода в ширину с помощью собственной очереди сделанной через структуру.

```
Выберите вариант работы:
1 - Генерировать граф
2 - Обход в ширину по матрице
3 - Обход в ширину по спискам смежности
4 - Обход в ширину по матрице с помощью очереди через список
5 - Сравнение скорости
0 - Выход
Ваш выбор: 4
1 2 4 3
```

2. Сравнение скорости времени работы.

```
Замер номер 1 на 100
Queue 0,008s:
ListQueue 0,007s:
Замер номер 2 на 150
Queue 0,011s:
ListQueue 0,012s:
Замер номер 3 на 200
Queue 0,016s:
ListQueue 0,016s:
Замер номер 4 на 250
Queue 0,021s:
ListQueue 0,023s:
Замер номер 5 на 300
Queue 0,030s:
ListQueue 0,025s:
```

Вывод

В ходе лабораторной работы были изучены матрица и списки смежности графа, реализованы генерация и BFS тремя способами. Обе матричные реализации имеют сложность $O(n^2)$, для n = 100–300 разница во времени незначительна; std::queue немного быстрее, списки смежности дают $O(n + m)$. Получены навыки работы с графами и структурами данных на C/C++.